INTEGRAL DRAGON

MODULE 3 • SESSION 09

Skills and Deterministic Hooks

Good advice is useful. Automatic enforcement is safer.

AI-Accelerated Software Development | Foundations Program

Confidential. Prepared exclusively for Ensemble Health Partners.

The model should not be your compliance memory

NOT ENGINEERING CONTROLS

- × "Remember to run tests before committing."
- × "Remember not to commit secrets to the repo."
- × "Remember to keep diffs small."
- × "Remember not to touch protected config files."

If a check is mandatory, the system should enforce it. Model memory is not a substitute for a deterministic control.

WHAT MODULE 3 CHANGES

- › Module 2 established advisory controls — specs, plans, CLAUDE.md
- › Session 09 is the first explicit move into **algorithmic controls**
- › Skills package reusable expertise that improves consistency
- › Hooks enforce mandatory behavior automatically at lifecycle events

Key distinction: Skills can improve consistency. Hooks *guarantee* a check ran. Enterprise teams need to know exactly where that boundary sits.

Skills and hooks solve different categories of work — treating them as substitutes creates fragility

SKILLS

- Reusable domain knowledge and guidance
- Architecture-specific implementation patterns
- Multi-step workflows that still benefit from judgment
- Structured review patterns for recurring tasks

What skills guarantee

Better-structured assistance when invoked. They do not guarantee the check happened — only that guidance was available.

VS

HOOKS

- Block unsafe commands before they run
- Auto-format after edits — always, not sometimes
- Run validations after file changes
- Protect specific files or paths from modification
- Trigger audit events and notifications

What hooks guarantee

The check ran. Regardless of what the model decided. The hook fires at the lifecycle event deterministically.

Hooks run at lifecycle boundaries — these are control points, not convenience triggers

`session-start`
Fires when a new session begins. Initialize context, load environment checks, verify prerequisites.

`prompt-submit`
Fires when a user prompt is submitted. Validate input, apply content filters, enforce scope constraints.

`pre-tool-use`
Fires before any tool call executes. Block dangerous commands, enforce path restrictions, require approval gates.

`post-tool-use`
Fires after a tool call completes. Run formatters, execute validators, trigger audit log writes, send notifications.

`stop`
Fires when a session or task ends. Final validation sweep, summary generation, cleanup tasks.

`subagent-start / stop`
Fires at subagent boundaries. Enforce delegation contracts, validate scope, gate output acceptance.

Scope matters here too: User scope for personal automation. Project scope for team-shared guardrails. Local scope for experimentation. Managed scope for organization policy. Team hooks belong in project scope — not in one engineer's local config.

Flexible guidance is not the same as mandatory enforcement — keep the distinction sharp

The question to ask

"Is this behavior negotiable?"

Yes, negotiable → Skill
Reusable guidance, domain knowledge, structured workflow help. Better outcomes when invoked, but judgment-based.

No, non-negotiable → Hook
Deterministic enforcement. The check must happen. The model does not get a vote on whether the hook fires.

Example policy split

Skill examples

"Review this API change against our service conventions"
"Apply our incident triage workflow to this alert"
"Analyze this PR for architecture boundary violations"

Hook examples

Block edits to protected config files (pre-tool-use)
Run formatter after every file write (post-tool-use)
Deny destructive shell commands (pre-tool-use)
Scan for secrets before commit (pre-tool-use)

LIVE DEMO — SESSION 09

Add one hook.
Watch advice become enforcement.

Same workflow. Same model. A hook fires — and the outcome is no longer probabilistic.

A strong hook policy reduces review burden — but overuse or misplacement creates its own problems

Demo flow

```
# Step 1: Without a hook
> Claude modifies a protected path
→ No enforcement. Behavior depends
      on whether CLAUDE.md was followed.

# Step 2: Add a project hook
# In settings.json (project scope):
{
  "hooks": [{
    "event": "pre-tool-use",
    "match": { "path": "/config/*" },
    "action": "block"
  }]
}

# Step 3: Same task — hook fires
→ Action blocked. Log written.
      Outcome is guaranteed, not hoped-for.
```

Common failure modes

Encoding mandatory policy only in CLAUDE.md

CLAUDE.md influences behavior. It does not guarantee it. Mandatory checks need hooks — not just written instructions the model might follow.

Overusing hooks for judgment-based tasks

Hooks that fire on everything create noise and friction without adding safety. Reserve hooks for genuinely non-negotiable checks.

Hiding team hooks in user-local settings

Shared guardrails belong in project scope so every teammate and CI agent benefits. A hook that only runs on one engineer's machine is not a team control.

SESSION 09 — KEY TAKEAWAY

Skills are reusable guidance. Hooks are deterministic enforcement.

Keep the distinction sharp. Enterprise delivery needs both — but they are not interchangeable.

Next: Session 10 — Once enforcement is in place, the next scaling problem is complexity. Session 10 breaks large work into isolated delegated tasks with subagents.